

Lab 23 - Events, Logs, Field Selectors and the Metrics Server

Field	Value
Lab id	Lab 23
Day / module	Day 5 - Observability
Duration	50 minutes
Namespace	kcna-lab23
Maps to	LO6 Implement regular monitoring of the Kubernetes system and perform necessary troubleshooting A6 Implement regular system reviews to monitor solution status and make modifications, according to an architecture management framework K7 Interactions among various IT components
Cluster	Single-node kind, Kubernetes v1.30+ (built and validated against v1.37.0)

How to read the expected-output blocks. Pod name suffixes, IP addresses,

AGE/LAST SEEN columns and timestamps are generated at runtime and **will

differ on your cluster**. Match the *shape*, the column headers, the event

reasons and the error strings.

>

Read the Part 5 warning before you start. kubectl top does not work

on a stock kind cluster. That section is explicitly conditional.

1. Objective

By the end of this lab you will be able to:

1. Query Events as first-class API objects, sort them meaningfully, and explain why the default listing order is misleading.
2. Filter Events with --field-selector on type, reason, involvedObject.kind and involvedObject.name, and state which fields are selectable and which are not.
3. Explain the Event TTL surprise - why an incident from two hours ago has no Events left - and name the mechanism that controls it.
4. Drive kubectl logs with -c, --previous, --since, --tail, -f and --prefix, and explain why only stdout/stderr is captured.
5. Explain why node-level log shipping exists by demonstrating a log that is invisible to kubectl logs until a sidecar makes it visible.
6. State precisely what kubectl top requires, install metrics-server on kind with the flag kind forces you to use, and explain why that flag would be a security defect in production.

2. Prerequisites

* A running single-node kind cluster.

```
kubectl version
```

```
Client Version: v1.37.0
Kustomize Version: v5.8.1
Server Version: v1.37.0
```

* You are in the lab directory:

```
cd courseware/labs/lab-23-events-logs-metrics
```

* Images used, all pinned: busybox:1.36, and - only if you choose to do Part 5 - registry.k8s.io/metrics-server/metrics-server:v0.7.2.

* Lab 22 is a soft prerequisite. You should already know what a Warning Unhealthy event looks like.

3. Scenario

The Meridian Freight post-incident review for INC-4417 stalled for an embarrassing reason: by the time the Depot Platform Squad opened the console the next morning, the Events were gone. The Slack thread contained screenshots of kubectl get pods and nothing else. Nobody could reconstruct the ordering of events, and the root cause had to be inferred.

The remediation ticket DEP-1019 says, in full:

```
*Build the evidence trail. Before the next incident we must be able to answer,
from the cluster itself: what did the control plane say, in what order; what
did the application print; and what was it consuming. Document what is
retained, for how long, and what is not retained at all.*
```

That is this lab.

4. Step-by-step procedure

Part 1 - Namespace and log corpus

Step 1.1 - Create the namespace.

```
kubectl apply -f manifests/00-namespace.yaml
```

```
namespace/kcna-lab23 created
```

Step 1.2 - Load the synthetic Depot Portal access log from data/ into a ConfigMap. This 30-line corpus is what the portal container will replay, so the log stream you analyse has realistic structure: INFO, WARN and ERROR lines, request ids, upstream timeouts and a rising queue depth.

```
kubectl create configmap depot-access-log \
  --from-file=depot-portal-access.log=data/depot-portal-access.log \
  -n kcna-lab23
```

```
configmap/depot-access-log created
```

Step 1.3 - Sanity-check the corpus you just loaded.

```
grep -c ERROR data/depot-portal-access.log
grep -c WARN data/depot-portal-access.log
```

```
6
5
```

Part 2 - Generate a real event stream

Step 2.1 - Apply the three workloads. Together they will produce Normal events (scheduling, pulling, starting), Warning events (FailedMount, BackOff) and a continuous log stream.

```
kubectl apply -f manifests/10-depot-portal-logger.yaml
kubectl apply -f manifests/20-flaky-scanner.yaml
kubectl apply -f manifests/30-missing-config.yaml
```

```
deployment.apps/depot-portal created
deployment.apps/gate-scanner created
pod/depot-gate-agent created
```

Step 2.2 - Wait about 60 seconds for the event stream to develop, then look at the Pods. Note the three different states: healthy, crash-looping, and stuck before it ever started.

```
kubectl get pods -n kcna-lab23
```

NAME	READY	STATUS	RESTARTS	AGE
depot-gate-agent	0/1	ContainerCreating	0	65s
depot-portal-58d6c47f9b-jm4vd	2/2	Running	0	65s
depot-portal-58d6c47f9b-rz8nq	2/2	Running	0	65s
gate-scanner-6c4f8b7d95-w2xkc	0/1	CrashLoopBackOff	2 (18s ago)	65s

depot-gate-agent sits in ContainerCreating forever, not Error. A

volume that cannot be mounted blocks the container from ever being created.

That distinction matters in Lab 25.

Part 3 - Events as first-class objects

Step 3.1 - List Events the naive way, and notice the problem.

```
kubectl get events -n kcna-lab23 | head -6
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
41s	Normal	Pulled	Pod/gate-scanner-6c4f8b7d95-w2xkc	Container image "busybox:1.36" already present on machine
2m6s	Normal	Scheduled	Pod/depot-portal-58d6c47f9b-jm4vd	Successfully assigned kcna-lab23/depot-portal-58d6c47f9b-jm4vd to kind-control-plane
19s	Warning	FailedMount	Pod/depot-gate-agent	MountVolume.SetUp failed for volume "gate-config" : configmap "depot-gate-config" not found
2m6s	Normal	Created	Pod/depot-portal-58d6c47f9b-jm4vd	Created container: portal
...				

The trap. Events are returned in the API's own order, which is *not*

chronological. During an incident this is actively harmful: you will

misread cause and effect.

Step 3.2 - Sort them. This is the form you should build muscle memory for.

```
kubectl get events -n kcna-lab23 --sort-by=.lastTimestamp | tail -12
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
2m30s	Normal	Scheduled	Pod/gate-scanner-6c4f8b7d95-w2xkc	Successfully assigned kcna-lab23/gate-scanner-6c4f8b7d95-w2xkc to kind-control-plane
2m28s	Normal	Pulled	Pod/gate-scanner-6c4f8b7d95-w2xkc	Container image "busybox:1.36" already present on machine
2m28s	Normal	Created	Pod/gate-scanner-6c4f8b7d95-w2xkc	Created container: scanner
2m28s	Normal	Started	Pod/gate-scanner-6c4f8b7d95-w2xkc	Started container scanner
2m16s	Warning	BackOff	Pod/gate-scanner-6c4f8b7d95-w2xkc	Back-off restarting failed container scanner in pod gate-scanner-6c4f8b7d95-w2xkc_kcna-lab23(...)
30s	Warning	FailedMount	Pod/depot-gate-agent	MountVolume.SetUp failed for volume "gate-config" : configmap "depot-gate-config" not found

Step 3.3 - The modern alternative. kubectl events (GA since v1.29) sorts by default and can scope to one object.

```
kubectl events -n kcna-lab23 --for pod/depot-gate-agent
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
3m1s	Normal	Scheduled	Pod/depot-gate-agent	Successfully assigned kcna-lab23/depot-gate-agent to kind-control-plane
2m59s (x2 over 3m1s)	Warning	FailedMount	Pod/depot-gate-agent	MountVolume.SetUp failed for volume "gate-config" : configmap "depot-gate-config" not found

Read the (x2 over 3m1s) aggregation. Kubernetes does not store one Event

per occurrence. Repeated identical events are collapsed into a single object

with a count and a firstTimestamp/lastTimestamp pair. Your "one" event

may represent hundreds of occurrences.

Step 3.4 - Field selectors. Filter to Warnings only - the single highest-value filter in day-to-day triage.

```
kubectl get events -n kcna-lab23 --field-selector type=Warning --sort-by=.lastTimestamp
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
2m40s	Warning	BackOff	Pod/gate-scanner-6c4f8b7d95-w2xkc	Back-off restarting failed container scanner in pod gate-scanner-6c4f8b7d95-w2xkc_kcna-lab23(...)
54s	Warning	FailedMount	Pod/depot-gate-agent	MountVolume.SetUp failed for volume "gate-config" : configmap "depot-gate-config" not found

Step 3.5 - Compound field selector: Warnings about Pods only. Comma means AND. In a real cluster this strips out Node, Deployment and PVC noise.

```
kubectl get events -n kcna-lab23 \
  --field-selector involvedObject.kind=Pod,type=Warning \
  --sort-by=.lastTimestamp
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
2m55s	Warning	BackOff	Pod/gate-scanner-6c4f8b7d95-w2xkc	Back-off restarting failed container scanner in pod gate-scanner-6c4f8b7d95-w2xkc_kcna-lab23(...)
69s	Warning	FailedMount	Pod/depot-gate-agent	MountVolume.SetUp failed for volume "gate-config" : configmap "depot-gate-config" not found

Step 3.6 - Select by reason, then by a specific object name.

```
kubectl get events -n kcna-lab23 --field-selector reason=FailedMount
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
84s	Warning	FailedMount	Pod/depot-gate-agent	MountVolume.SetUp failed for volume "gate-config" : configmap "depot-gate-config" not found

```
kubectl get events -n kcna-lab23 \
  --field-selector involvedObject.name=depot-gate-agent,involvedObject.kind=Pod
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
3m30s	Normal	Scheduled	Pod/depot-gate-agent	Successfully assigned kcna-lab23/depot-gate-agent to kind-control-plane
89s	Warning	FailedMount	Pod/depot-gate-agent	MountVolume.SetUp failed for volume "gate-config" : configmap "depot-gate-config" not found

Step 3.7 - Discover the limits of field selectors. Not every field is indexed for selection - you cannot select on arbitrary paths, and you cannot use label selectors on Events at all (Events carry no meaningful labels).

```
kubectl get events -n kcna-lab23 --field-selector message=whatever
```

```
Error from server (BadRequest): Unable to find "/v1, Resource=events" that match label selector "", field selector "message=whatever": field label not supported: message
```

The rule. For core v1 Events the selectable fields are

involvedObject.kind, involvedObject.namespace, involvedObject.name,

involvedObject.uid, involvedObject.apiVersion,

involvedObject.resourceVersion, involvedObject.fieldPath, reason,

reportingComponent, source, type and metadata.name/metadata.namespace.

Anything else - including message - must be filtered client-side with grep

or jq.

Step 3.8 - Look at a raw Event object to see the fields you have been selecting on, plus the two that explain aggregation.

```
kubectl get events -n kcna-lab23 \
  --field-selector reason=FailedMount -o yaml | \
  grep -E '^ (count|firstTimestamp|lastTimestamp|reason|type)|^ (kind|name):'
```

```
kind: Pod
name: depot-gate-agent
count: 3
firstTimestamp: "2026-09-05T06:12:04Z"
lastTimestamp: "2026-09-05T06:16:21Z"
reason: FailedMount
type: Warning
```

Step 3.9 - The Event TTL surprise. Events are stored in etcd like any other object, but the kube-apiserver garbage-collects them on a timer controlled by its --event-ttl flag. The default is one hour. Confirm the setting your cluster is actually running - read-only inspection of the static Pod manifest on the kind node:

```
docker exec kind-control-plane \
  grep -E 'event-ttl' /etc/kubernetes/manifests/kube-apiserver.yaml
```

(no output)

No output is the expected result and it is the lesson. kubeadm does not

set --event-ttl, so the kube-apiserver default of 1 hour applies. If

your control-plane node has a different name, get it with

`kubectl get nodes -o name`.

Confirm the effective default from the running process:

```
kubectl -n kube-system get pod -l component=kube-apiserver \
  -o jsonpath='{.items[0].spec.containers[0].command}' | tr ',' '\n' | grep -c event-ttl
```

0

Consequences you must be able to state. Events are not an audit log.

They expire, they are aggregated rather than enumerated, and they are

namespaced. Anything you need after an hour must be shipped somewhere else -

which is exactly why the Kubernetes Events API is a *signal source* for an

observability pipeline, not the pipeline itself. This is the argument for

Lab 24.

Step 3.10 - Fill in rows 3.2 through 3.7 of data/event-triage-worksheet.csv with what each query returned.

Part 4 - Logs

Step 4.1 - The Pod has two containers, so kubectl logs refuses to guess.

```
POD=$(kubectl get pods -n kcna-lab23 -l app=depot-portal \
-o jsonpath='{.items[0].metadata.name}')
kubectl logs -n kcna-lab23 "$POD"
```

error: a container name must be specified for pod depot-portal-58d6c47f9b-jm4vd, choose one of:
[portal audit-sidecar]

Step 4.2 - Select the container with -c, and bound the output with --tail.

```
kubectl logs -n kcna-lab23 "$POD" -c portal --tail=6
```

```
2026-09-05T06:00:41Z INFO depot-portal gate=sg-tuas lane=inbound req_id=e41533 method=GET
path=/api/v1/shipments status=200 duration_ms=40 driver=DRV-2210
2026-09-05T06:00:43Z INFO depot-portal gate=sg-tuas lane=inbound req_id=f52644 method=GET
path=/api/v1/manifest/SG44140 status=200 duration_ms=57 driver=DRV-2210
2026-09-05T06:00:45Z INFO depot-portal gate=sg-tuas lane=outbound req_id=063755 method=POST
path=/api/v1/dispatch status=201 duration_ms=198 driver=DRV-9042
2026-09-05T06:00:47Z INFO depot-portal gate=sg-tuas lane=inbound req_id=174866 method=GET
path=/healthz status=200 duration_ms=1
2026-09-05T06:00:49Z WARN depot-portal gate=sg-tuas lane=inbound req_id=285977 method=GET
path=/api/v1/manifest/SG44151 status=404 duration_ms=11 reason=manifest_not_indexed
2026-09-05T06:00:51Z INFO depot-portal gate=sg-tuas lane=inbound req_id=396a88 method=POST
path=/api/v1/checkin status=201 duration_ms=122 driver=DRV-1187
2026-09-05T06:00:53Z ERROR depot-portal gate=sg-tuas lane=outbound req_id=4a7b99 method=POST
path=/api/v1/dispatch status=502 duration_ms=3006 upstream=manifest-indexer
error=upstream_timeout
```

The timestamps in the log body are from the corpus, not from now. That is

deliberate: application timestamps and cluster wall-clock time are different

things, and mixing them up is a real source of confusion in incident reviews.

Ask the *cluster* for its view of time with --timestamps.

Step 4.3 - Time-bound the query with --since, and add cluster-side timestamps. This is the combination you want when you know roughly when something happened.

```
kubectl logs -n kcna-lab23 "$POD" -c portal --since=30s --timestamps | tail -3
```

```
2026-09-05T06:19:12.884301Z 2026-09-05T06:00:31Z ERROR depot-portal gate=sg-tuas lane=outbound
req_id=9fc0ee method=POST path=/api/v1/dispatch status=502 duration_ms=3009 upstream=manifest-
indexer error=upstream_timeout
2026-09-05T06:19:14.886117Z 2026-09-05T06:00:33Z INFO depot-portal gate=sg-tuas lane=inbound
req_id=a0d1ff method=POST path=/api/v1/checkin status=201 duration_ms=131 driver=DRV-3341
2026-09-05T06:19:16.888445Z 2026-09-05T06:00:35Z WARN depot-portal gate=sg-tuas lane=outbound
req_id=b1e200 method=GET path=/api/v1/queue status=200 duration_ms=1204 note=queue_depth_high
depth=58
```

Two timestamps per line: the first is when the container runtime received the line, the second is what the application wrote.

Step 4.4 - Aggregate across replicas with a label selector and --prefix. This is how you read a Deployment rather than a Pod.

```
kubectl logs -n kcna-lab23 -l app=depot-portal -c portal --tail=2 --prefix
```

```
[pod/depot-portal-58d6c47f9b-jm4vd/portal] 2026-09-05T06:00:53Z ERROR depot-portal gate=sg-
tuas lane=outbound req_id=4a7b99 method=POST path=/api/v1/dispatch status=502 duration_ms=3006
upstream=manifest-indexer error=upstream_timeout
[pod/depot-portal-58d6c47f9b-jm4vd/portal] 2026-09-05T06:00:55Z INFO depot-portal gate=sg-
tuas lane=inbound req_id=5b8caa method=GET path=/api/v1/shipments status=200 duration_ms=36
driver=DRV-3341
[pod/depot-portal-58d6c47f9b-rz8nq/portal] 2026-09-05T06:00:19Z INFO depot-portal gate=sg-
tuas lane=inbound req_id=3f6a88 method=GET path=/api/v1/shipments status=200 duration_ms=38
driver=DRV-9042
[pod/depot-portal-58d6c47f9b-rz8nq/portal] 2026-09-05T06:00:21Z INFO depot-portal gate=sg-
tuas lane=inbound req_id=4a7b99 method=POST path=/api/v1/checkin status=201 duration_ms=127
driver=DRV-1187
```

Step 4.5 - --previous, the single most important logs flag in triage. The gate-scanner container is crash-looping. Its *current* container may have only just started, so the reason it died is in the *terminated* container's log.

```
SCANNER=$(kubectl get pods -n kcna-lab23 -l app=gate-scanner \
-o jsonpath='{.items[0].metadata.name}')
kubectl logs -n kcna-lab23 "$SCANNER" --previous
```

```
06:18:02 INFO gate-scanner starting, serial=SCN-0447
06:18:02 INFO opening barcode reader on /dev/ttyUSB0
06:18:07 WARN reader handshake retry 1/1
06:18:12 ERROR reader handshake failed: device not present
06:18:12 FATAL gate-scanner exiting with status 1
```

Compare with the current container, which is mid-boot or backing off:

```
kubectl logs -n kcna-lab23 "$SCANNER"
```

```
06:19:44 INFO gate-scanner starting, serial=SCN-0447
06:19:44 INFO opening barcode reader on /dev/ttyUSB0
```

Only one generation back is kept. --previous gives you the immediately

preceding container, not a history. If a Pod restarts twice while you are

making coffee, the original failure is gone.

Step 4.6 - Confirm the exit code that matches that log, from the API rather than the log text.

```
kubectl get pod -n kcna-lab23 "$SCANNER" \
-o jsonpath='{.status.containerStatuses[0].lastState.terminated.exitCode}{"\n"}{.status.
containerStatuses[0].lastState.terminated.reason}{"\n"}'
```

```
1
Error
```

Step 4.7 - Why node-level log shipping exists. The audit-sidecar container writes audit records to a file, and separately tails that file to stdout. First, look at the file directly - this is what the application actually produced:

```
kubectl exec -n kcna-lab23 "$POD" -c audit-sidecar -- tail -3 /var/log/depot/audit.log
```

```
2026-09-05T06:19:30Z AUDIT seq=14 actor=gate-scanner action=scan result=accepted
2026-09-05T06:19:35Z AUDIT seq=15 actor=gate-scanner action=scan result=accepted
2026-09-05T06:19:40Z AUDIT seq=16 actor=gate-scanner action=scan result=accepted
```

Now look at what the cluster can see:

```
kubectl logs -n kcna-lab23 "$POD" -c audit-sidecar --tail=3
```

```
2026-09-05T06:19:30Z AUDIT seq=14 actor=gate-scanner action=scan result=accepted
2026-09-05T06:19:35Z AUDIT seq=15 actor=gate-scanner action=scan result=accepted
2026-09-05T06:19:40Z AUDIT seq=16 actor=gate-scanner action=scan result=accepted
```

They match - but only because a tail -f process is deliberately re-emitting the file to stdout. Delete that sidecar behaviour and kubectl logs shows nothing at all, while the file keeps filling up inside the container and disappears the moment the Pod is deleted.

Step 4.8 - Confirm where the container runtime actually stores what it captured, on the node.

```
docker exec kind-control-plane sh -c 'ls /var/log/pods/kcna-lab23_'"$POD"'_*/portal/'
```

```
0.log
```

```
docker exec kind-control-plane sh -c \
'cat /etc/kubernetes/kubelet.conf >/dev/null && echo readable'
```

```
readable
```

The three facts that justify a log agent.

1. Only stdout/stderr is captured. Files inside the container are not.
2. The kubelet rotates those files - by default at 10Mi per file with 5 files retained (containerLogMaxSize / containerLogMaxFiles in the kubelet configuration). Older lines are deleted, not archived.
3. Everything is node-local and Pod-lifetime-bound. Delete the Pod and kubectl logs returns NotFound immediately.

>

A node-level agent such as Fluent Bit or Fluentd (both CNCF projects; Fluentd is CNCF-graduated) runs as a DaemonSet, tails /var/log/pods/... and ships lines off-node before rotation destroys them. That is the entire reason the pattern exists.

Step 4.9 - Prove fact 3.

```
kubectl delete pod -n kcna-lab23 "$POD"
kubectl logs -n kcna-lab23 "$POD" -c portal --tail=1
```

```
pod "depot-portal-58d6c47f9b-jm4vd" deleted
Error from server (NotFound): pods "depot-portal-58d6c47f9b-jm4vd" not found
```

The Deployment immediately replaces it, but the deleted Pod's logs are unrecoverable.

Step 4.10 - Record rows 4.3 and 4.5 in data/event-triage-worksheet.csv.

Part 5 - The Metrics Server (CONDITIONAL)

Read this box before running anything in Part 5

>

kubectl top does not work on a stock kind cluster. Resource metrics are served by the metrics.k8s.io API, which is provided by the **metrics-server** add-on. kind does not install it. This is not a broken cluster.

>

You have two valid ways to complete this part:

>

* Path A - install metrics-server using the manifests in this lab. This creates four cluster-scoped objects and **one** additive RoleBinding in kube-system, all listed and justified in the header comments of manifests/90-metrics-server.yaml and manifests/91-metrics-server-kube-system-rolebinding.yaml.

* Path B - skip the install. Run Steps 5.1 and 5.2 to see and understand the failure, then read Steps 5.4-5.7 without executing them. Parts 1-4 are the assessed content and are unaffected.

>

No kubectl top numbers are printed anywhere in this lab as if they were observed on your machine. Where output is shown for Path A, the numeric columns are marked as varying.

Step 5.1 - Establish the starting state. Ask for metrics before installing anything.

```
kubectl top pods -n kcna-lab23
```

```
error: Metrics API not available
```

Step 5.2 - Confirm **why**, from the API discovery layer. This is the diagnostic that distinguishes "add-on missing" from "add-on broken".

```
kubectl get apiservice v1beta1.metrics.k8s.io
```

```
Error from server (NotFound): apiservices.apiregistration.k8s.io "v1beta1.metrics.k8s.io" not found
```

```
kubectl api-resources --api-group=metrics.k8s.io
```

NAME	SHORTNAMES	APIVERSION	NAMESPACED	KIND
------	------------	------------	------------	------

An empty table. The API group does not exist.

Stop here if you are taking Path B. Everything below installs software.

Step 5.3 (Path A) - Read the two manifest headers *first*. They are the teaching content of this part, not boilerplate.

```
sed -n '1,60p' manifests/90-metrics-server.yaml
sed -n '1,35p' manifests/91-metrics-server-kube-system-rolebinding.yaml
```

Then apply, main bundle first:

```
kubectl apply -f manifests/90-metrics-server.yaml
```

```
serviceaccount/metrics-server created
clusterrole.rbac.authorization.k8s.io/kcna-lab23:metrics-server created
clusterrolebinding.rbac.authorization.k8s.io/kcna-lab23:metrics-server created
clusterrolebinding.rbac.authorization.k8s.io/kcna-lab23:metrics-server:auth-delegator created
service/metrics-server created
deployment.apps/metrics-server created
apiservice.apiregistration.k8s.io/v1beta1.metrics.k8s.io created
```

```
kubectl apply -f manifests/91-metrics-server-kube-system-rolebinding.yaml
```

```
rolebinding.rbac.authorization.k8s.io/kcna-lab23-metrics-server-auth-reader created
```

Step 5.4 (Path A) - Wait for it to become Ready. Allow up to 90 seconds: the readiness probe has a 20 second initial delay and the first scrape cycle must complete.

```
kubectl rollout status deployment/metrics-server -n kcna-lab23 --timeout=120s
```

```
deployment "metrics-server" successfully rolled out
```

```
kubectl get apiservice v1beta1.metrics.k8s.io
```

NAME	SERVICE	AVAILABLE	AGE
v1beta1.metrics.k8s.io	kcna-lab23/metrics-server	True	75s

AVAILABLE True is the gate. If it reads False (FailedDiscoveryCheck), see the Troubleshooting table.

Step 5.5 (Path A) - Now kubectl top works.

```
kubectl top nodes
```

NAME	CPU(cores)	CPU(%)	MEMORY(bytes)	MEMORY(%)
kind-control-plane	<varies>	<varies>	<varies>	<varies>

The numbers are deliberately not written out. They depend entirely on your

host machine, what else is running on it, and how long metrics-server has been

collecting. Anyone who prints specific figures here is inventing them. What you must verify is that five columns appear and the node is named - that proves the aggregation layer is working.

Step 5.6 (Path A) - Give it something predictable to measure. Apply the CPU consumer, which is capped by a 100m CPU limit.

```
kubectl apply -f manifests/40-load-generator.yaml
kubectl rollout status deployment/manifest-indexer -n kcna-lab23 --timeout=60s
```

```
deployment.apps/manifest-indexer created
deployment "manifest-indexer" successfully rolled out
```

Wait ~60 seconds for two scrape cycles, then:

```
kubectl top pods -n kcna-lab23 --containers
```

POD	READY	NAME	CPU(cores)	MEMORY(bytes)
manifest-indexer-...	1/1	indexer	<near 100m>	<small>
depot-portal-...	2/2	portal	<near 0m>	<small>
depot-portal-...	2/2	audit-sidecar	<near 0m>	<small>
metrics-server-...	1/1	metrics-server	<varies>	<varies>

What you can legitimately predict, and why. The indexer container runs an unbounded arithmetic loop, so it will consume every cycle it is permitted. Its limits.cpu is 100m, which the kernel enforces as a CFS quota. It should therefore report at or just below 100m - and crucially, *never above it*.

That is a claim about the enforcement mechanism, which is deterministic. The exact millicore reading is not.

>

Confirm the ceiling yourself:

```
kubectl get pod -n kcna-lab23 -l app=manifest-indexer \
-o jsonpath='{.items[0].spec.containers[0].resources.limits}{"\n"}'
```

```
{"cpu": "100m", "memory": "64Mi"}
```

Step 5.7 (Path A) - See the raw API underneath kubectl top. top is only a formatter.

```
kubectl get --raw '/apis/metrics.k8s.io/v1beta1/namespaces/kcna-lab23/pods' \
| head -c 400; echo
```

```
{
  "kind": "PodMetricsList",
  "apiVersion": "metrics.k8s.io/v1beta1",
  "metadata": {},
  "items": [
    {
      "metadata": {
        "name": "manifest-indexer-...",
        "namespace": "kcna-lab23",
        "creationTimestamp": "...",
        "timestamp": "...",
        "window": "15.0s",
        "containers": [
          {
            "name": "indexer",
            "usage": {
              "cpu": "...",
              "memory": "..."
            }
          }
        ]
      }
    }
  ]
}
```

Three properties to note, and remember for Lab 24. The response carries a

window (a short sampling interval), it has no history at all, and it stores nothing. metrics-server keeps a small in-memory ring buffer to serve the current value for the HorizontalPodAutoscaler and for kubectl top. It is not a monitoring system, it cannot answer "what was CPU at 03:00 last night", and it must never be used as one. A time-series database - Prometheus - is what fills that gap, and that is Lab 24.

Step 5.8 - Record row 5.4 in data/event-triage-worksheet.csv, noting which path (A or B) you took.

5. Verification

```
bash verification/checks.sh
```

The script auto-detects whether you took Path A or Path B and adjusts. See verification/expected-output.md.

6. Failure injection (mandatory)

You will now break the metrics pipeline in the way it most commonly breaks in the field: the aggregation layer points at a Service that has no healthy backend. If you took Path B, do the alternative injection in Step 6.6 instead.

Step 6.1 (Path A) - Scale metrics-server to zero. The APIService still exists, so the API group is still **advertised** - it just cannot be served.

```
kubectl scale deployment metrics-server -n kcna-lab23 --replicas=0
```

```
deployment.apps/metrics-server scaled
```

Step 6.2 - Wait ~30 seconds, then observe the failure mode. This is a **different** error string from Step 5.1, and the difference is the whole point.

```
kubectl top pods -n kcna-lab23
```

```
Error from server (ServiceUnavailable): the server is currently unable to handle the request (
get pods.metrics.k8s.io)
```

Step 6.3 - Diagnose from the aggregation layer.

```
kubectl get apiservice v1beta1.metrics.k8s.io
```

NAME	SERVICE	AVAILABLE	AGE
v1beta1.metrics.k8s.io	kcna-lab23/metrics-server	False (MissingEndpoints)	6m

```
kubectl get apiservice v1beta1.metrics.k8s.io \
-o jsonpath='{.status.conditions[0].message}'
```

```
endpoints for service/metrics-server in "kcna-lab23" have no addresses with port name "https"
```

Learn both signatures.

error: Metrics API not available → the APIService does not exist; the add-on was never installed.

Error from server (ServiceUnavailable) + AVAILABLE: False → the APIService exists but its backend is down. Completely different fixes.

Step 6.4 - Repair.

```
kubectl scale deployment metrics-server -n kcna-lab23 --replicas=1
kubectl rollout status deployment/metrics-server -n kcna-lab23 --timeout=120s
```

```
deployment.apps/metrics-server scaled
deployment "metrics-server" successfully rolled out
```

Step 6.5 - Confirm recovery.

```
kubectl get apiservice v1beta1.metrics.k8s.io
```

NAME	SERVICE	AVAILABLE	AGE
v1beta1.metrics.k8s.io	kcna-lab23/metrics-server	True	8m

Step 6.6 (Path B alternative - no install required) - Break the log evidence trail instead, which is the failure that actually cost Meridian its INC-4417 post-mortem. Force a second restart of the crash-looping scanner and watch the original evidence become unrecoverable.

```
kubectl get pods -n kcna-lab23 -l app=gate-scanner \
-o jsonpath='{.items[0].status.containerStatuses[0].restartCount}{"\n"}'
kubectl delete pod -n kcna-lab23 -l app=gate-scanner
```

```
4
pod "gate-scanner-6c4f8b7d95-w2xkc" deleted
```

```
NEW=$(kubectl get pods -n kcna-lab23 -l app=gate-scanner \
-o jsonpath='{.items[0].metadata.name}')
kubectl logs -n kcna-lab23 "$NEW" --previous
```

```
Error from server (BadRequest): previous terminated container "scanner" in pod "gate-scanner-...
." not found
```

That is the INC-4417 failure, reproduced. A brand-new Pod has no previous container, so there is no evidence at all. Combined with the one-hour Event TTL from Step 3.9, an incident that is not investigated promptly becomes uninvestigable. Ship your logs and events off-cluster.

7. Troubleshooting

Symptom	Likely cause	Diagnostic command	Fix	
error: Metrics API not available	metrics-server is not installed; stock kind has no metrics.k8s.io	kubectl get apiservice v1beta1.metrics.k8s.io returns NotFound	Install it (Part 5 Path A), or accept it and take Path B	
AVAILABLE: False (FailedDiscoveryCheck) on the APIService, metrics-server Pod is 0/1 and its log shows x509: cannot validate certificate ... doesn't contain any IP SANs	--kubelet-insecure-tls is missing; kind kubelets serve self-signed certificates	`kubectl logs -n kcna-lab23 deploy/metrics-server \	tail -20`	Add --kubelet-insecure-tls (already present in the supplied manifest - check you applied the supplied file)
metrics-server Pod runs but readiness never passes, log shows unable to load configmap based request-header-client-ca-file or configmaps "extension-apiserver-authentication" is forbidden	The kube-system RoleBinding was not applied	kubectl get rolebinding kcna-lab23-metrics-server-auth-reader -n kube-system	Apply manifests/91-metrics-server-kube-system-rolebinding.yaml, or take Path B	
Error from server (ServiceUnavailable) from kubectl top	APIService exists but has no healthy backend	kubectl get apiservice v1beta1.metrics.k8s.io -o jsonpath='{.status.conditions[0].message}'	Scale/repair the metrics-server Deployment; check its Service targetPort name is https	
field label not supported: message	You tried to field-select on a non-indexed Event field	-	Field-select on type/reason/involvedObject.* only; filter message text client-side with grep	
Events you expected are simply absent	kube-apiserver --event-ttl (default 1 hour) garbage-collected them	`kubectl get events -n <ns> --sort-by=.lastTimestamp \	head -1` shows nothing older than ~1h	Nothing to fix in-cluster. Ship Events to durable storage if you need them
kubectl logs says a container name must be specified	Multi-container Pod	kubectl get pod <pod> -o jsonpath='{.spec.containers[*].name}'	Add -c <container>, or --all-containers	
previous terminated container ... not found	The Pod was recreated, or has never restarted	kubectl get pod <pod> -o jsonpath='{.status.containerStatuses[0].restartCount}'	Only one generation is retained; capture logs promptly or ship them off-node	
Application writes to a file and kubectl logs shows nothing	Only stdout/stderr is captured by the container runtime	kubectl exec <pod> -c <c> -- tail /path/to/file	Log to stdout, or add a sidecar that tails the file (as audit-sidecar does here)	

8. Cleanup

Step 8.1 - Delete the lab namespace. This removes every namespaced object including metrics-server itself.

```
kubectl delete namespace kcna-lab23
```

```
namespace "kcna-lab23" deleted
```

Step 8.2 - Path A only. The cluster-scoped objects and the kube-system RoleBinding are **not** in the namespace and survive Step 8.1. Remove them by exact name - never with a wildcard or a bare kubectl delete:

```
kubectl delete apiservice v1beta1.metrics.k8s.io
kubectl delete clusterrolebinding kcna-lab23:metrics-server
kubectl delete clusterrolebinding kcna-lab23:metrics-server:auth-delegator
kubectl delete clusterrole kcna-lab23:metrics-server
kubectl delete rolebinding kcna-lab23:metrics-server-auth-reader -n kube-system
```

```
apiservice.apiregistration.k8s.io "v1beta1.metrics.k8s.io" deleted
clusterrolebinding.rbac.authorization.k8s.io "kcna-lab23:metrics-server" deleted
clusterrolebinding.rbac.authorization.k8s.io "kcna-lab23:metrics-server:auth-delegator" deleted
clusterrole.rbac.authorization.k8s.io "kcna-lab23:metrics-server" deleted
rolebinding.rbac.authorization.k8s.io "kcna-lab23:metrics-server-auth-reader" deleted
```

****If Lab 24 or a later exercise needs kubectl top, leave metrics-server**

installed and run Step 8.2 at the end of the day instead.** Note that leaving

a dangling APIService whose Service no longer exists will make

kubectl get apiservice report False and can slow down kubectl api-resources

for everyone on the cluster - so if you delete the namespace, delete the

APIService too.

Step 8.3 - Verify.

```
kubectl get namespace kcna-lab23
kubectl get apiservice v1beta1.metrics.k8s.io
```

```
Error from server (NotFound): namespaces "kcna-lab23" not found
Error from server (NotFound): apiservices.apiregistration.k8s.io "v1beta1.metrics.k8s.io" not found
```

9. What you learned

* Events are API objects with all the properties of API objects - and two surprising ones. They are aggregated (count + firstTimestamp), so one row can mean hundreds of occurrences; and they are garbage-collected after --event-ttl, one hour by default. Events are a live triage signal, never an audit trail.

* --sort-by=.lastTimestamp is not optional. The default listing order is not chronological and will make you misread causality during an incident.

* Field selectors are server-side and cheap, but restricted. type, reason and involvedObject.* are indexed; message is not. Knowing the boundary saves you from piping the whole namespace through grep.

* kubectl logs sees stdout/stderr and nothing else, keeps exactly one previous container generation, and is bounded by kubelet log rotation (10Mi × 5 files by default). Those three limits, together, are the complete argument for a node-level shipper such as Fluent Bit or Fluentd.

* kubectl top is an add-on, not a feature. It requires metrics-server serving metrics.k8s.io through the aggregation layer. On kind it also requires --kubelet-insecure-tls, because kind kubelets serve self-signed certificates - a flag that disables TLS verification and would be a genuine security defect in production, where the fix is signed kubelet serving certificates instead.

* metrics-server has no history. It answers "now", in a ~15 s window, for the HPA and for kubectl top. Any question containing the word "yesterday" needs a time-series database. That is Lab 24.

* Two failure signatures, two different fixes: Metrics API not available (never installed) versus ServiceUnavailable + AVAILABLE: False (installed, backend down).

Further reading

* Kubernetes - *Application Introspection and Debugging*:

<<https://kubernetes.io/docs/tasks/debug/debug-application/debug-running-pod/>>

* Kubernetes - *Logging Architecture* (node-level agents, rotation, sidecars):

<<https://kubernetes.io/docs/concepts/cluster-administration/logging/>>

* Kubernetes - *Resource Metrics Pipeline*:

<<https://kubernetes.io/docs/tasks/debug/debug-cluster/resource-metrics-pipeline/>>

* Kubernetes - *Field Selectors*:

<<https://kubernetes.io/docs/concepts/overview/working-with-objects/field-selectors/>>

* metrics-server - README, including the --kubelet-insecure-tls caveat:

<<https://github.com/kubernetes-sigs/metrics-server>>

* Fluentd (CNCF graduated) and Fluent Bit: <<https://www.cncf.io/projects/fluentd/>>

* CNCF - KCNA curriculum, *Cloud Native Architecture → Observability*: <<https://github.com/cncf/curriculum>>